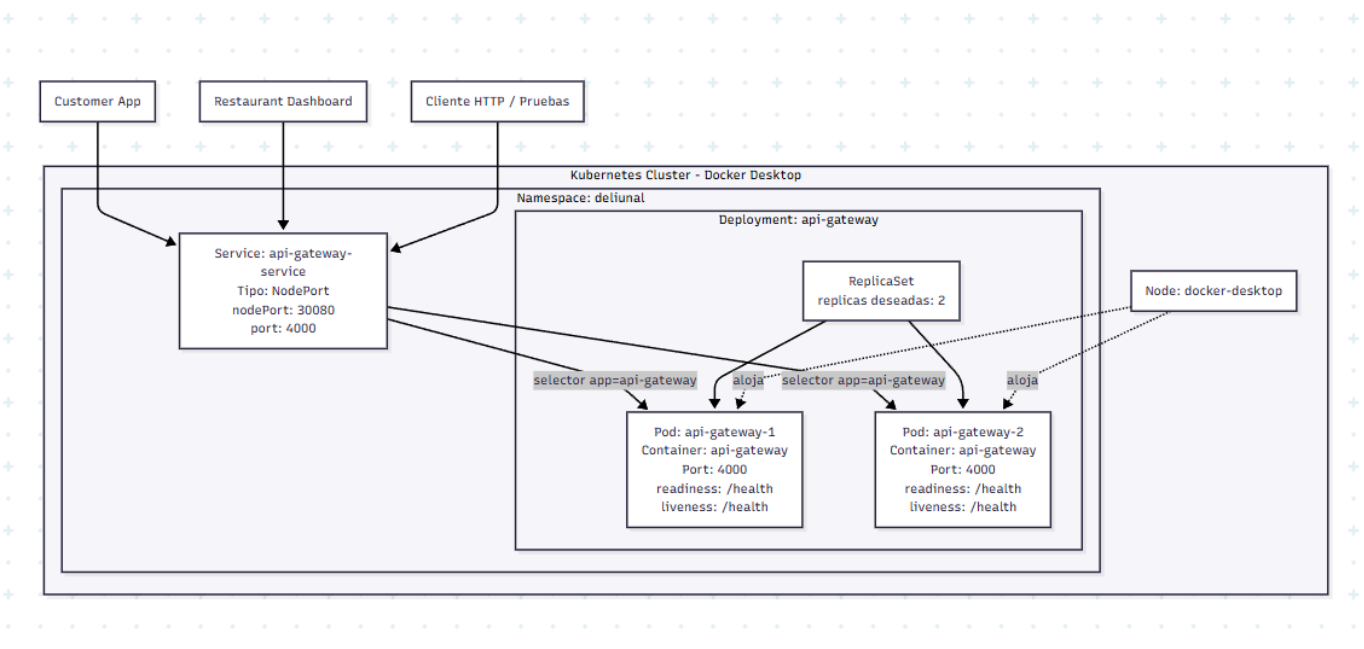


Laboratory 6 — Reliability

(2026-I)

- Manuel Alejandro Navas Bohorquez
- German Camilo Bernal Ladino
- Juan David Rivera Buitrago
- Obed Felipe Espinosa Angarita
- Edwin Felipe Pinilla Peralta

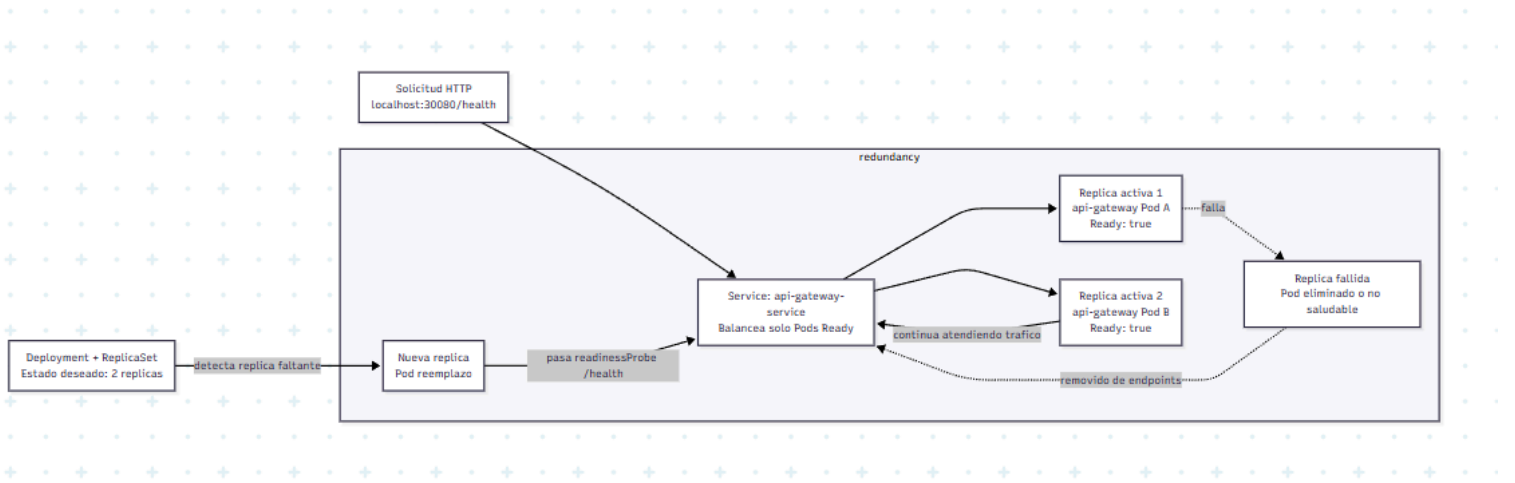
1. Vista arquitectónica del Cluster Pattern (nodos, Pods, Services, réplicas).



- El Cluster Pattern se aplica usando Kubernetes como orquestador. El clúster agrupa los recursos que ejecutan el api-gateway, mantiene el número deseado de réplicas y expone el servicio mediante una abstracción estable. En este prototipo el tipo de clúster es local de un nodo, ejecutado con Kubernetes de Docker Desktop

Elemento	Función arquitectónica	Configuración aplicada
Nodo	Capacidad de cómputo donde Kubernetes agenda los Pods.	Clúster local de Docker Desktop, un nodo para demostración.
Namespace	Aísla recursos del proyecto DELIUNAL.	deliunal.
Deployment	Declara el estado deseado y delega en ReplicaSet la creación de Pods.	api-gateway con replicas: 2.
Pods	Instancias ejecutables del API Gateway.	Dos Pods equivalentes, stateless, con puerto 4000.
Service	Punto de acceso estable y balanceo hacia Pods listos.	NodePort 30080 hacia targetPort 4000.
Probes	Controlan disponibilidad y recuperación automática.	readinessProbe y livenessProbe sobre /health.

2. Vista arquitectónica del patrón de redundancia aplicado.



Atributo de calidad	Disponibilidad y tolerancia a fallos del punto de entrada HTTP.
Fuente del estímulo	Falla de un Pod del API Gateway o eliminación manual durante la demostración.
Artefacto afectado	Deployment/api-gateway, Pods y Service/api-gateway-service.
Respuesta esperada	El Service conserva al menos una réplica disponible y Kubernetes restaura el número deseado de réplicas.
Medida de respuesta	/health responde 200; el Deployment vuelve a 2/2 réplicas listas.

3. Guía técnica Parte A: descripción del patrón, tipo de clúster implementado, pasos de implementación, manifiestos de Kubernetes utilizados, y evidencia de self-healing y escalado.

Descripción del patrón

El Cluster Pattern organiza varios recursos de cómputo como una unidad lógica. En Kubernetes, esa unidad se expresa mediante objetos declarativos: un Deployment define el estado deseado, un ReplicaSet mantiene las réplicas, los Pods ejecutan los contenedores y un Service ofrece una dirección estable para acceder a ellos. Tipo de clúster implementado Se implementa un clúster local de Kubernetes en Docker Desktop, adecuado para laboratorio y demostración. El clúster tiene un solo nodo, pero permite validar abstracciones propias de Kubernetes: despliegues declarativos, réplicas, descubrimiento por Service, probes, self-healing y escalado manual. Pasos de implementación

1. Verificar Docker Desktop con Kubernetes habilitado y contexto apuntando al clúster local.
2. Construir la imagen del API Gateway: `docker build -t deliunal-api-gateway:k8s ./services/api-gateway`.
3. Aplicar los manifiestos: `kubectl apply -f infrastructure/kubernetes/`.
4. Esperar el despliegue: `kubectl rollout status deployment/api-gateway -n deliunal`.
5. Validar Deployment, Pods, Service y endpoints.
6. Ejecutar pruebas de self-healing y escalado.

Fragmento del Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-gateway
  namespace: deliunal
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api-gateway
  template:
    metadata:
      labels:
        app: api-gateway
    spec:
      containers:
        - name: api-gateway
          image: deliunal-api-gateway:k8s
          ports:
            - name: http
              containerPort: 4000
```

Fragmento del Service

```
apiVersion: v1
kind: Service
metadata:
  name: api-gateway-service
  namespace: deliunal
spec:
  type: NodePort
  selector:
    app: api-gateway
  ports:
    - name: http
      port: 4000
      targetPort: 4000
      nodePort: 30080
```

Evidencia de self-healing

La evidencia se obtiene eliminando una réplica y observando que Kubernetes crea otra para volver al estado deseado. En la estación donde se generó este PDF no estaba disponible `kubect1`, por lo que se documentan los comandos y la salida esperada para reproducir la evidencia en el clúster.

```
kubect1 get pods -n deliunal -l app=api-gateway
```

NAME	READY	STATUS	RESTARTS	AGE
api-gateway-xxxxxxxxx-aaaaa	1/1	Running	0	2m
api-gateway-xxxxxxxxx-bbbbb	1/1	Running	0	2m

```
kubect1 delete pod api-gateway-xxxxxxxxx-aaaaa -n deliunal
```

```
kubect1 get pods -n deliunal -l app=api-gateway -w
```

NAME	READY	STATUS	RESTARTS	AGE
api-gateway-xxxxxxxxx-bbbbb	1/1	Running	0	2m
api-gateway-yyyyyyyyyy-cccccc	0/1	ContainerCreating	0	4s
api-gateway-yyyyyyyyyy-cccccc	1/1	Running	0	18s

4. Guía técnica Parte B: descripción del patrón de redundancia, escenario de calidad, pasos de implementación, fragmentos de código o configuración relevantes, evidencia de failover y recomendaciones.

Descripción del patrón

La redundancia aplicada es activa: existen dos instancias equivalentes del API Gateway ejecutándose en paralelo. Ambas aceptan el mismo contrato HTTP y están registradas detrás del mismo Service. La capacidad hot spare existe porque la instancia adicional ya está levantada y lista, no requiere aprovisionamiento después de la falla.

Escenario de calidad

Escenario	Un Pod del API Gateway falla durante operación normal.
Estímulo	El proceso deja de responder a <code>/health</code> o el Pod es eliminado.
Ambiente	Tráfico HTTP activo hacia el Service <code>api-gateway-service</code> .
Respuesta	El Service evita enviar tráfico al Pod no listo y el ReplicaSet crea una nueva réplica.
Métrica	Disponibilidad del endpoint <code>/health</code> y recuperación a 2/2 réplicas listas.

Pasos de implementación

1. Seleccionar un componente stateless y crítico de entrada: `api-gateway`.
2. Configurar `replicas`: 2 para tener redundancia activa desde el inicio.
3. Agregar `readiness probe` para que solo las réplicas sanas reciban tráfico.
4. Agregar `liveness probe` para reiniciar instancias bloqueadas o no saludables.
5. Exponer el conjunto de réplicas por medio de un único Service.
6. Probar failover eliminando un Pod y verificando continuidad del endpoint.

Evidencia de failover

El failover se valida verificando que el Service conserve endpoints listos y que el endpoint de salud responda mientras una réplica está siendo reemplazada.

```
kubectl get endpoints api-gateway-service -n deliunal
```

NAME	ENDPOINTS	AGE
api-gateway-service	10.1.0.12:4000,10.1.0.13:4000	3m

```
kubectl delete pod api-gateway-xxxxxxxxx-aaaaa -n deliunal
```

Invoke-WebRequest http://localhost:30080/health

```
StatusCode      : 200
Content         : {"status":"ok"}
```

```
kubectl rollout status deployment/api-gateway -n deliunal
```

deployment "api-gateway" successfully rolled out

Recomendaciones

- Ejecutar el clúster productivo con más de un nodo para tolerar fallas de nodo, no solo fallas de Pod.
- Agregar `resources.requests` y `resources.limits` para que el scheduler tenga criterios de capacidad.
- Usar `HorizontalPodAutoscaler` para escalar por CPU, memoria o métricas de tráfico.
- Definir `PodDisruptionBudget` para conservar al menos una réplica disponible durante mantenimientos.
- Separar `readiness` y `liveness` si el chequeo de disponibilidad requiere dependencias externas.
- Usar un `LoadBalancer` o `Ingress Controller` para exposición real fuera de laboratorio.
- Conservar el API Gateway `stateless`; sesiones, caché o estado compartido deben vivir en servicios externos.